GUIDE PERSO

Piloter Claude Code depuis Discord

Comment le projet `claude-discord` est construit, comment le refaire de zéro, et comment partager ton serveur avec d'autres personnes.

Sommaire

1. Ce que fait le projet
2. Ce qu'il te faut
3. Comment c'est construit (les pièces)
4. Le refaire étape par étape
5. Comment ça marche à l'intérieur
6. Les commandes
7. Partager le serveur avec d'autres
8. Sécurité : les règles à garder
9. Dépannage

Dossier du projet : `C:\Myfiles\Logiciels\claude-discord` · Octobre 2026

1. Ce que fait le projet

Tu écris à Claude dans Discord (au téléphone, à la voix, où tu veux), et c'est **Claude Code sur ton PC** qui fait le travail : il lit et modifie tes fichiers, lance des commandes, puis te répond dans Discord.

Ton serveur Discord devient une copie de la barre latérale de l'app Claude :

```
📁 Projet Remaster Silver          ← catégorie = un projet (un dossier du PC)
   # game-decompilation            ← salon = une conversation avec Claude
   # wsl-disk-space-analysis
📁 Sans dossier
   # apex-movement-mechanics
📦 Archives
# accueil                          ← l'aide et les commandes du bot
```

Dans chaque salon, le déroulé est toujours le même :

```
Toi : ajoute un bouton pause
Bot : ⌛ Patience, je traite ta requête…
Bot : (la réponse, coupée en messages de 2000 caractères max)
Bot : ✅ Réponse terminée (42 s) — tu peux envoyer ta prochaine question.
      🪙 40,3 k lus (dont 32 k en cache) · 850 écrits
      ⏱️ 5 h : reste 81 % · 🗓️ semaine : reste 39 %
```

Le bot reprend aussi les conversations que tu as eues dans l'app Claude (import et synchro automatique toutes les 10 minutes) et renomme les salons comme l'app.

2. Ce qu'il te faut

- **Un PC Windows allumé** : c'est lui qui travaille. S'il dort, le bot ne répond plus.
- **Claude Code** installé et connecté à ton abonnement Claude (la commande `claude` doit marcher dans un terminal).
- **Node.js 24 ou plus** : le bot n'a besoin d'aucune bibliothèque, il utilise le WebSocket et le `fetch` intégrés à Node.
- **Un compte Discord** et un serveur à toi.

> Pas besoin de `npm install` : trois fichiers `.mjs` (environ 1000 lignes en tout) suffisent.

3. Comment c'est construit

Fichier	Rôle
`bridge.mjs`	Le bot lui-même (~790 lignes). Il se connecte à Discord, reçoit tes messages, lance Claude, renvoie la réponse, gère les catégories et salons, les commandes `!…` et la synchro avec l'app.
`sessions.mjs`	Lit les conversations de l'app Claude stockées sur le PC (`~\.claude\projects\…\*.jsonl`) : titre, dossier, derniers échanges.
`split.mjs`	Découpe une longue réponse en morceaux de 2000 caractères max, sans casser les blocs de code.
`config.json`	Tes réglages et le **token secret** du bot. Ne le partage jamais.
`config.example.json`	Le modèle de config, sans secret.
`state.json`	La mémoire du bot : quel salon correspond à quelle conversation et à quel dossier. Créé tout seul.
`start.bat`	Lance le bot et le relance après 5 s s'il s'arrête.
`attachments\`	Les fichiers et captures que tu envoies dans Discord, pour que Claude puisse les lire.

4. Le refaire étape par étape

Étape 1 — Créer ton serveur

Dans Discord : **+** → **Créer le mien** → **Pour moi et mes amis**.

Étape 2 — Créer le bot

Un webhook ne sait qu'*envoyer* des messages, pas les lire. Il faut donc un vrai bot (gratuit).

1. Va sur `discord.com/developers/applications` → **New Application** → donne-lui un nom (ex. « MyClaude »).
2. Onglet **Bot** :
   - **Reset Token** → copie le token. C'est le mot de passe du bot.
   - Active **Message Content Intent**. Sans ça, le bot voit qu'il y a un message mais pas son texte.
   - Décoche **Public Bot** : personne d'autre ne pourra l'ajouter à son serveur.
3. Onglet **Installation** : mets le lien d'installation sur **None**. Sinon Discord refuse de passer le bot en privé.
4. Onglet **OAuth2** → **URL Generator** : coche le scope `bot`, puis la permission **Administrator**. Ouvre l'URL générée et choisis ton serveur.

> Si Discord affiche *« Integration requires code grant »*, va dans l'onglet **Bot** et désactive **Requires OAuth2 Code Grant**.

Étape 3 — Récupérer les identifiants

Paramètres utilisateur → **Avancés** → **Mode développeur** (activer). Ensuite :

- clic droit sur l'icône du serveur → **Copier l'identifiant du serveur** → c'est le `guildId` ;
- clic droit sur ton pseudo → **Copier l'identifiant de l'utilisateur** → c'est le `ownerId`.

Étape 4 — Configurer

Copie `config.example.json` en `config.json` et remplis-le :

Clé	Rôle
`discordToken`	le token du bot (secret)
`guildId`	l'ID de ton serveur : le bot ignore tous les autres
`ownerId`	ton ID : **seuls tes messages** sont envoyés à Claude
`claudePath`	chemin de `claude.exe`
`model` / `effort`	`claude-opus-5-5` / `medium` par défaut
`permissionMode`	`auto` conseillé : Claude fait seul ce qui est sûr et refuse ce qui est risqué
`maxParallel`	conversations qui peuvent travailler en même temps (3)
`timeoutMinutes`	durée max d'une requête (60)
`showUsage` / `showCost`	affiche les tokens et le forfait restant / le coût
`syncMinutes`, `syncDays`, `syncMaxNew`, `importHistory`	réglages de la synchro avec l'app (toutes les 10 min, sur 7 jours, 15 nouvelles conversations max, 5 échanges repris)
`appendSystemPrompt`	consignes ajoutées à Claude : réponses courtes, pas de tableaux, dictée vocale…

Étape 5 — Lancer

Double-clic sur `start.bat`. La fenêtre doit rester ouverte. Au premier lancement, le bot crée le salon **#accueil**.

Pour qu'il démarre avec Windows : `Win + R` → `shell:startup` → mets-y un raccourci vers `start.bat`.

Étape 6 — Premier test

1. Dans #accueil : `!projet C:\chemin\de\ton\projet` → une catégorie et un salon `nouvelle-conversation` apparaissent.
2. Écris une question dans ce salon : tu dois voir ⏳, la réponse, puis ✅.
3. `!importer` pour récupérer tes conversations de l'app.

5. Comment ça marche à l'intérieur

La connexion à Discord

Le bot ouvre une connexion permanente (la *Gateway*, un WebSocket) avec Discord. Discord y pousse chaque nouveau message en temps réel. Toutes les ~40 s, le bot envoie un « battement de cœur » pour dire qu'il est vivant. Si la connexion tombe, il se reconnecte et reprend là où il en était.

Pour *envoyer* des messages, créer ou renommer des salons, il utilise l'API web de Discord (des requêtes HTTP classiques). Quand Discord dit « trop vite » (erreur 429), le bot attend le délai demandé puis réessaie.

Le trajet d'un message

1. Discord pousse ton message au bot.
2. Le bot vérifie que c'est **ton serveur** et **toi**. Sinon il l'ignore.
3. Il marque le salon « occupé » et envoie ⏳ Patience.
4. Il lance Claude Code en arrière-plan dans le dossier du projet :

   ```
   claude -p --resume <id-conversation> --model claude-opus-5-5 --effort medium
          --permission-mode auto --output-format stream-json --verbose
   ```

   Ta question passe par l'entrée standard. `--resume` fait que Claude garde le fil de la conversation.
5. Claude travaille et écrit sa progression ligne par ligne (format JSON). La dernière ligne contient la réponse, l'ID de la conversation et les tokens. D'autres lignes donnent le forfait restant (fenêtres de 5 h et de la semaine).
6. `split.mjs` découpe la réponse en messages de 2000 caractères max. Si un bloc de code est coupé, il le referme et le rouvre dans le message suivant.
7. Le bot envoie les morceaux, puis ✅ et les lignes de conso en petit.
8. Après la première réponse, un petit appel à Haiku génère un titre court et renomme le salon.

La synchro avec l'app Claude

L'app Claude enregistre chaque conversation dans un fichier `.jsonl` dans `C:\Users\<toi>\.claude\projects\`. Certains fichiers dépassent 100 Mo, donc le bot n'en lit que le début (pour le dossier) et la fin (titre, derniers échanges).

Toutes les 10 minutes, ou avec `!synchro`, il :

- recopie dans Discord les échanges faits dans l'app depuis la dernière fois ;
- importe les nouvelles conversations qui ont un titre dans l'app ;
- renomme les salons dont le titre a changé, et les déplace si la conversation a changé de dossier.

Il ignore ce qui vient de lui-même (les conversations lancées en `claude -p`, déjà dans Discord), les interruptions et les échanges encore en cours.

Redémarrage

`!redemarrer` arrête le bot proprement et `start.bat` le relance. La commande est refusée si une requête est en cours, pour ne pas couper une réponse.

6. Les commandes

Commande	Effet
`!projet C:\dossier`	crée la catégorie du projet et un premier salon
`!projets`	liste les projets et leurs dossiers
`!importer` puis `!importer 1 3` ou `!importer tout`	recrée tes conversations de l'app en salons
`!synchro`	met tout à jour depuis l'app tout de suite
`!nouveau [titre]`	nouvelle conversation dans le même projet
`!stop`	annule la requête en cours dans ce salon
`!status`	libre ou occupé, dossier, conversation
`!conso`	forfait restant (5 h et semaine)
`!archiver`	range le salon dans 📦 Archives
`!redemarrer`	relance le bot
`!aide`	rappelle la liste

Tu peux aussi créer un salon à la main dans une catégorie de projet : il devient une nouvelle conversation. Une pièce jointe (capture, fichier) est téléchargée sur le PC et Claude la lit.

7. Partager le serveur avec d'autres

Tu peux inviter des amis pour qu'ils **lisent** tes conversations avec Claude et en discutent avec toi. Le bot, lui, continue de n'obéir qu'à toi. Leurs messages sont ignorés, c'est voulu (voir la partie 8).

Inviter quelqu'un

1. Clic droit sur l'icône du serveur → **Inviter des gens**.
2. **Modifier le lien d'invitation** : choisis une expiration (ex. 1 jour) et un nombre d'utilisations (ex. 1). Un lien qui traîne pourrait laisser entrer n'importe qui.
3. Envoie le lien à la personne.

Choisir ce qu'ils voient : un rôle « Invité »

Par défaut, tout membre voit tous les salons. Pour contrôler ça proprement :

1. **Paramètres du serveur** → **Rôles** → **Créer un rôle**, nomme-le « Invité ».
2. Donne ce rôle à tes amis (clic droit sur leur pseudo → Rôles).
3. Sur chaque **catégorie** privée (clic droit → Modifier la catégorie → Permissions) :
   - pour `@everyone` : **Voir les salons** ✗ ;
   - et si tu veux la montrer à tes invités : ajoute le rôle « Invité » avec **Voir les salons** ✓.
4. Les salons d'une catégorie suivent ses permissions (« synchronisés »). Les nouveaux salons créés par le bot dans cette catégorie sont donc couverts automatiquement.

> **Plus simple :** crée une catégorie « Partagé » où tu ranges les conversations à montrer. Seule cette catégorie est visible du rôle « Invité ». Tout le reste reste caché.

Lecture seule ou discussion ?

- **Lecture seule :** sur la catégorie, rôle « Invité » → **Envoyer des messages** ✗. Ils lisent sans pouvoir écrire.
- **Pour discuter entre vous :** crée un salon `#discussion` hors des catégories de projets. Le bot n'y crée rien, et comme il ne t'y répond pas, ça reste un salon de discussion normal.

> **Attention :** si tu écris toi-même un message dans un salon de projet, le bot l'envoie à Claude. Pour parler à tes amis, utilise `#discussion` .

Ce que tes invités ne doivent jamais avoir

- **Administrateur** ou **Gérer les salons** : ils pourraient supprimer ou déplacer des salons et casser l'organisation du bot.

- **Gérer le serveur** ou **Gérer les rôles**.

Et s'ils veulent leur propre Claude dans Discord ?

Le plus propre : chacun installe ce projet chez lui, avec **son** PC, **son** abonnement Claude et **son** bot. Les abonnements Claude sont personnels, donc ne fais pas tourner ton bot pour d'autres. Vous pouvez quand même être sur le même serveur Discord : chaque bot a un `ownerId` différent et ne répond qu'à son propriétaire. Donnez-leur des catégories séparées pour ne pas mélanger les conversations.

8. Sécurité : les règles à garder

> Ton bot donne à Claude un accès réel à ton PC : fichiers, commandes, internet. Quiconque pourrait lui parler pourrait agir sur ton PC.

- **Garde `ownerId` = ton ID uniquement.** Ne l'élargis jamais à d'autres personnes.
- **Le token du bot est un mot de passe.** Ne le colle nulle part : ni dans Discord, ni dans une capture, ni dans un dépôt Git. S'il fuit : Developer Portal → Bot → **Reset Token**, puis mets le nouveau dans `config.json`.
- **Bot privé** : « Public Bot » décoché, pour que personne ne puisse l'inviter ailleurs.
- **Ce que tes invités voient, ils le voient en entier** : le code, les chemins de fichiers, parfois des infos perso. Ne partage que les catégories sans données sensibles.
- Garde `permissionMode` sur `auto`. `bypassPermissions` laisse Claude tout faire sans filet.
- Active l'**authentification à deux facteurs** sur ton compte Discord : si on te le vole, on peut parler au bot en se faisant passer pour toi.

9. Dépannage

Problème	Solution
Le bot est hors ligne	Le PC dort ou `start.bat` est fermé. Relance-le.
Erreur 4014 au démarrage	Message Content Intent n'est pas activé dans le portail.
Erreur 4004	Token invalide : régénère-le et mets-le à jour.
« Integration requires code grant »	Onglet Bot → décoche Requires OAuth2 Code Grant.
Le bot ne répond pas à un ami	Normal : il ne répond qu'à `ownerId`.
« Attends le ✅ »	Une requête tourne déjà dans ce salon. Attends ou fais `!stop`.
Une conversation de l'app n'apparaît pas	`!synchro`. Seules celles qui ont un titre dans l'app sont importées automatiquement. Sinon `!importer`.
Claude dit qu'une action est bloquée	Le mode `auto` a refusé une action risquée. Fais-la sur le PC.
Tu as modifié le code du bot	`!redemarrer`.

Pense aussi à ne pas écrire dans la même conversation depuis l'app et depuis Discord en même temps. Les conversations créées depuis Discord se reprennent sur le PC avec `claude --resume` dans le dossier du projet.